PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT

CASE STUDY REPORT

Submitted by: B. Lakshman

Roll Number: 26000916

Branch: CSM-1

College: DR.RVR NRI DTBU

Academic Year: 2026–30

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Calculate a household electricity bill using consumption slabs, a fixed charge, a surcharge, and a rebate. Compare bills for several months and identify an unusual increase.

2. Objectives

The main objectives of this case study are:

1. To calculate electricity charges using progressive consumption slabs.
2. To apply fixed charges, surcharge and rebate rules.
3. To compare monthly bills and identify unusual increases.

3. Assumptions

The following assumptions are considered:

1. Consumption units are non-negative whole numbers.
2. The slab rates, fixed charge, surcharge and rebate are predefined in the program.
3. A bill increase above 30 percent is treated as unusual.

4. Algorithm

1. Read the number of months.
2. For each month, read the month name and units consumed.
3. Calculate the energy charge using progressive slabs.
4. Add the fixed charge.
5. Apply a surcharge when consumption is above 500 units.
6. Apply a rebate when consumption is below 100 units.

7. Store and display every bill.
8. Compare each month with the previous month and report an increase above 30%.

5. Pseudocode

```
START
READ number_of_months
FOR each month
    READ month_name and units
    IF units ⩽ 100, calculate first-slab charge
    ELSE IF units ⩽ 300, calculate first and second slab charges
    ELSE, calculate first, second and third slab charges
    ADD fixed charge
    IF units > 500, ADD surcharge
    IF units < 100, SUBTRACT rebate
    STORE bill
END FOR
FOR every month after the first
    percentage_increase = (current - previous) / previous * 100
    IF percentage_increase > 30, report unusual increase
END FOR
DISPLAY bills
STOP
```

6. Python Source Code

```
def energy_charge(units):
    if units ⩽ 100:
        charge = units * 2
    elif units ⩽ 300:
        charge = 100 * 2 + (units - 100) * 4
    else:
        charge = 100 * 2 + 200 * 4 + (units - 300) * 6
    return charge


def calculate_bill(units):
    fixed_charge = 100
    surcharge = 0
    rebate = 0

    bill = energy_charge(units) + fixed_charge

    if units > 500:
        surcharge = bill * 10 / 100
        bill = bill + surcharge

    if units < 100:
        rebate = 25
        bill = bill - rebate

    return bill


months = int(input("Enter number of months: "))
month_names = []
bills = []

for i in range(months):
    name = input("Enter month name: ")
    units = int(input("Enter units consumed: "))
    bill = calculate_bill(units)
    month_names.append(name)
    bills.append(bill)

print("\nMonthly Bills")
for i in range(months):
```

```
    print(month_names[i], ": Rs.", round(bills[i], 2))

for i in range(1, months):
    if bills[i - 1] != 0:
        increase = (bills[i] - bills[i - 1]) * 100 / bills[i - 1]
        if increase > 30:
            print("Unusual increase from", month_names[i - 1], "to", monthl
```

7. Test Cases

S.No.	Test condition	Expected result
1	Units below 100	Bill includes rebate
2	Units between 101 and 300	First and second slabs applied
3	Units above 500	Surcharge applied

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Enter number of months: 3
Enter month name: January
Enter units consumed: 80
Enter month name: February
Enter units consumed: 220
Enter month name: March
Enter units consumed: 600

Monthly Bills
January : Rs. 235
February : Rs. 780
March : Rs. 3190.0
Unusual increase from January to February
Unusual increase from February to March
```

9. Conclusion

The Progressive Electricity Billing was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 2: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime and incentive. Apply progressive deductions and display a salary statement.

2. Objectives

The main objectives of this case study are:

1. To calculate gross salary from different earning components.
2. To apply progressive deductions.
3. To display a formatted salary statement.

3. Assumptions

The following assumptions are considered:

1. All salary inputs are non-negative numbers.
2. Overtime pay equals overtime hours multiplied by the hourly overtime rate.
3. The deduction slabs are fixed according to the problem definition.

4. Algorithm

1. Read employee name, basic pay, allowances, overtime hours, hourly overtime rate and incentive.
2. Calculate overtime pay.
3. Calculate gross salary.
4. Apply a 5% deduction up to Rs. 30,000, 10% on the next Rs. 20,000 and 15% above Rs. 50,000.
5. Calculate net salary.

6. Display the salary statement.

5. Pseudocode

```
START
READ employee details
 overtime_pay = overtime_hours * overtime_rate
 gross = basic + allowance + overtime_pay + incentive
 IF gross ⩽ 30000, deduction = gross * 5%
 ELSE IF gross ⩽ 50000, deduction = 30000 * 5% + remaining * 10%
 ELSE, deduction = 30000 * 5% + 20000 * 10% + remaining * 15%
 net = gross - deduction
DISPLAY salary statement
STOP
```

6. Python Source Code

```
def calculate_deduction(gross):
    if gross ⩽ 30000:
        deduction = gross * 5 / 100
    elif gross ⩽ 50000:
        deduction = 30000 * 5 / 100
        deduction = deduction + (gross - 30000) * 10 / 100
    else:
        deduction = 30000 * 5 / 100
        deduction = deduction + 20000 * 10 / 100
        deduction = deduction + (gross - 50000) * 15 / 100
    return deduction


name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
allowance = float(input("Enter allowances: "))
overtime_hours = float(input("Enter overtime hours: "))
overtime_rate = float(input("Enter overtime rate per hour: "))
incentive = float(input("Enter incentive: "))

overtime_pay = overtime_hours * overtime_rate
gross = basic + allowance + overtime_pay + incentive
deduction = calculate_deduction(gross)
net = gross - deduction

print("\nSALARY STATEMENT")
print("Employee:", name)
print("Basic Pay:", round(basic, 2))
print("Allowances:", round(allowance, 2))
print("Overtime Pay:", round(overtime_pay, 2))
print("Incentive:", round(incentive, 2))
print("Gross Salary:", round(gross, 2))
print("Deduction:", round(deduction, 2))
print("Net Salary:", round(net, 2))
```

7. Test Cases

S.No.	Test condition	Expected result
1	Gross salary Rs. 25,000	5% deduction applied
2	Gross salary Rs. 45,000	First two deduction slabs applied
3	Gross salary above Rs. 50,000	All progressive slabs applied

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
SALARY STATEMENT
Employee: Anu
Basic Pay: 42000.0
Allowances: 6000.0
Overtime Pay: 2400.0
Incentive: 1500.0
Gross Salary: 51900.0
Deduction: 5735.0
Net Salary: 46165.0
```

9. Conclusion

The Employee Payroll and Deduction Analysis was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 3: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Process multiple products, quantities, category-specific tax rates and threshold discounts. Limit the total discount to a maximum concession.

2. Objectives

The main objectives of this case study are:

1. To process products, quantities and category-specific taxes.
2. To apply threshold discounts with a maximum concession.
3. To generate an itemized invoice.

3. Assumptions

The following assumptions are considered:

1. Product prices and quantities are positive.
2. The categories are food, clothing or electronics.
3. The maximum discount concession is Rs. 1,000.

4. Algorithm

1. Read the number of products.
2. For every product, read name, category, unit price and quantity.
3. Calculate item amount.
4. Apply category tax: food 5%, clothing 12%, electronics 18%.
5. Calculate subtotal and tax total.
6. Apply 10% discount if subtotal is at least Rs. 5,000.

7. Limit discount to Rs. 1,000.
8. Display the invoice.

5. Pseudocode

```
START
READ number_of_products
subtotal = 0, total_tax = 0
FOR every product
    READ name, category, price and quantity
    amount = price * quantity
    ADD amount to subtotal
    SELECT tax rate according to category
    ADD amount * tax rate to total_tax
END FOR
IF subtotal ⩾ 5000, discount = subtotal * 10%
ELSE discount = 0
IF discount > 1000, discount = 1000
final_amount = subtotal + total_tax - discount
DISPLAY invoice totals
STOP
```

6. Python Source Code

```
def tax_rate(category):
    if category == "food":
        return 5
    elif category == "clothing":
        return 12
    elif category == "electronics":
        return 18
    else:
        return 0


number = int(input("Enter number of products: "))
subtotal = 0
all_tax = 0
names = []
amounts = []

for i in range(number):
    name = input("Enter product name: ")
    category = input("Enter category (food/clothing/electronics): ")
    price = float(input("Enter unit price: "))
    quantity = int(input("Enter quantity: "))

    amount = price * quantity
    tax = amount * tax_rate(category) / 100
    subtotal = subtotal + amount
    all_tax = all_tax + tax
    names.append(name)
    amounts.append(amount + tax)

if subtotal ⩾ 5000:
    discount = subtotal * 10 / 100
else:
    discount = 0

if discount > 1000:
    discount = 1000

final_amount = subtotal + all_tax - discount

print("\nITEMIZED INVOICE")
```

```
for i in range(number):
    print(names[i], ": Rs.", round(amounts[i], 2))
print("Subtotal: Rs.", round(subtotal, 2))
print("Tax: Rs.", round(all_tax, 2))
print("Discount: Rs.", round(discount, 2))
print("Final Amount: Rs.", round(final_amount, 2))
```

7. Test Cases

S.No.	Test condition	Expected result
1	Subtotal below Rs. 5,000	No threshold discount
2	Subtotal at least Rs. 5,000	10% discount applied
3	Discount above Rs. 1,000	Discount limited to Rs. 1,000

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
ITEMIZED INVOICE
Notebook : Rs. 1050.0
Shirt : Rs. 2240.0
Headphones : Rs. 3540.0
Subtotal: Rs. 6500.0
Tax: Rs. 830.0
Discount: Rs. 650.0
Final Amount: Rs. 6680.0
```

9. Conclusion

The Retail Billing with Multiple Discount Rules was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional

statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 4: EXAMINATION RESULT AND GRACE-MARK VALIDATION

1. Problem Statement

Process marks in multiple subjects. A student passes a subject with at least 40 marks. A maximum of 5 grace marks may be used only when the original mark is at least 35. Grace marks must not exceed the maximum mark.

2. Objectives

The main objectives of this case study are:

1. To process marks in multiple subjects.
2. To apply the specified grace-mark policy without exceeding maximum marks.
3. To generate the final classification.

3. Assumptions

The following assumptions are considered:

1. The pass mark is 40 for every subject.
2. Grace marks can be used only when the mark is at least 35 and the shortage is at most 5.
3. The final mark cannot exceed the supplied maximum mark.

4. Algorithm

1. Read the number of subjects and maximum marks.
2. Read each subject name and mark.
3. If mark is at least 40, mark the subject as passed.
4. If mark is between 35 and 39, use the required grace marks, up to 5.
5. Do not increase a mark beyond the maximum mark.

6. Calculate total and percentage.
7. Classify the result only if all subjects are passed.

5. Pseudocode

```
START
READ number_of_subjects and maximum_marks
passed_all = TRUE
FOR each subject
    READ subject and mark
    IF mark ⩾ 40
        final_mark = mark
    ELSE IF mark ⩾ 35 AND 40 - mark ⩽ 5
        final_mark = mark + (40 - mark)
    ELSE
        final_mark = mark
        passed_all = FALSE
    IF final_mark > maximum_marks, final_mark = maximum_marks
    ADD final_mark to total
END FOR
percentage = total / total_maximum * 100
IF passed_all is FALSE, classification = FAIL
ELSE classify using percentage
DISPLAY result
STOP
```

6. Python Source Code

```
number = int(input("Enter number of subjects: "))
maximum = int(input("Enter maximum mark for each subject: "))
subjects = []
final_marks = []
total = 0
passed_all = True

for i in range(number):
    subject = input("Enter subject name: ")
    mark = int(input("Enter mark: "))
    final_mark = mark
    grace = 0

    if mark ⩾ 40:
        print("Passed without grace marks")
    elif mark ⩾ 35:
        grace = 40 - mark
        if grace ⩽ 5:
            final_mark = mark + grace
            print("Grace marks used:", grace)
        else:
            passed_all = False
    else:
        passed_all = False

    if final_mark > maximum:
        final_mark = maximum

    if final_mark < 40:
        passed_all = False

    subjects.append(subject)
    final_marks.append(final_mark)
    total = total + final_mark

percentage = total * 100 / (number * maximum)

if passed_all == False:
    classification = "FAIL"
elif percentage ⩾ 75:
    classification = "DISTINCTION"
```

```
elif percentage ⩾ 60:
    classification = "FIRST CLASS"
elif percentage ⩾ 50:
    classification = "SECOND CLASS"
else:
    classification = "PASS"

print("\nRESULT")
for i in range(number):
    print(subjects[i], ":", final_marks[i])
print("Total:", total)
print("Percentage:", round(percentage, 2))
print("Classification:", classification)
```

7. Test Cases

S.No.	Test condition	Expected result
1	Mark 40 or above	Pass without grace
2	Mark 35 to 39	Required grace marks applied
3	Mark below 35	Subject remains failed

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Grace marks used: 3
RESULT
Python : 40
Mathematics : 72
English : 81
Total: 193
Percentage: 64.33
Classification: FIRST CLASS
```

9. Conclusion

The Examination Result and Grace-Mark Validation was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 5: ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Process deposits and withdrawals, maintain the account balance, reject invalid transactions, enforce a maximum withdrawal limit, and produce a summary.

2. Objectives

The main objectives of this case study are:

1. To simulate deposits and withdrawals.
2. To maintain the account balance after each transaction.
3. To reject invalid transactions and summarize the transaction sequence.

3. Assumptions

The following assumptions are considered:

1. The opening balance is supplied by the user.
2. Only D and W transaction types are accepted.
3. The maximum permitted withdrawal per transaction is Rs. 10,000.

4. Algorithm

1. Read the opening balance and number of transactions.
2. For each transaction, read its type and amount.
3. Accept positive deposits.
4. Accept a withdrawal only when it is positive, within the transaction limit, and not greater than the balance.
5. Reject all invalid transactions.

6. Count successful deposits, successful withdrawals and rejected transactions.
7. Display the final balance and summary.

5. Pseudocode

```
START
READ opening_balance and number_of_transactions
balance = opening_balance
FOR every transaction
    READ type and amount
    IF amount ⩽ 0, reject
    ELSE IF type is deposit, add amount
    ELSE IF type is withdrawal
        IF amount > withdrawal_limit, reject
        ELSE IF amount > balance, reject
        ELSE subtract amount
    ELSE reject
END FOR
DISPLAY balance and transaction counts
STOP
```

6. Python Source Code

```
balance = float(input("Enter opening balance: "))
number = int(input("Enter number of transactions: "))
withdrawal_limit = 10000
successful_deposits = 0
successful_withdrawals = 0
rejected = 0

for i in range(number):
    transaction_type = input("Enter transaction type (D/W): ")
    amount = float(input("Enter amount: "))

    if amount ⩽ 0:
        print("Rejected: amount must be positive")
        rejected = rejected + 1
    elif transaction_type == "D":
        balance = balance + amount
        successful_deposits = successful_deposits + 1
        print("Deposit successful")
    elif transaction_type == "W":
        if amount > withdrawal_limit:
            print("Rejected: withdrawal limit exceeded")
            rejected = rejected + 1
        elif amount > balance:
            print("Rejected: insufficient balance")
            rejected = rejected + 1
        else:
            balance = balance - amount
            successful_withdrawals = successful_withdrawals + 1
            print("Withdrawal successful")
    else:
        print("Rejected: invalid transaction type")
        rejected = rejected + 1

print("\nTRANSACTION SUMMARY")
print("Successful deposits:", successful_deposits)
print("Successful withdrawals:", successful_withdrawals)
print("Rejected transactions:", rejected)
print("Final balance: Rs.", round(balance, 2))
```

7. Test Cases

S.No.	Test condition	Expected result
1	Positive deposit	Balance increases
2	Withdrawal within balance	Withdrawal accepted
3	Withdrawal above balance	Transaction rejected

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Deposit successful
Withdrawal successful
Rejected: insufficient balance
Rejected: withdrawal limit exceeded

TRANSACTION SUMMARY
Successful deposits: 1
Successful withdrawals: 1
Rejected transactions: 2
Final balance: Rs. 8500.0
```

9. Conclusion

The ATM Transaction Sequence Simulation was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 6: ADVANCED NUMERIC PATTERN GENERATOR

1. Problem Statement

Generate a hollow diamond pattern for a positive odd dimension. Reject even or non-positive dimensions.

2. Objectives

The main objectives of this case study are:

1. To validate the requested pattern dimension.
2. To generate a hollow symmetric diamond using nested loops.
3. To practise pattern generation using conditions and loops.

3. Assumptions

The following assumptions are considered:

1. The dimension must be a positive odd number.
2. An asterisk represents the boundary of the diamond.
3. Spaces are used to align the pattern.

4. Algorithm

1. Read the dimension.
2. Check that it is positive and odd.
3. Generate the upper half using nested loops.
4. Generate the lower half in reverse order.
5. Print stars only at the boundary positions to create a hollow diamond.

5. Pseudocode

```
START
READ n
IF n ⩽ 0 OR n is even
    DISPLAY invalid dimension
ELSE
    middle = n / 2
    FOR row from 0 to middle
        print leading spaces
        FOR column from 0 to 2*row
            print star when column is first or last, otherwise space
    FOR row from middle-1 down to 0
        repeat the same boundary logic
STOP
```

6. Python Source Code

```
n = int(input("Enter an odd positive dimension: "))

if n <= 0 or n % 2 == 0:
    print("Invalid dimension. Enter a positive odd number.")
else:
    middle = n // 2

    for row in range(middle + 1):
        spaces = middle - row
        width = 2 * row + 1
        print(" " * spaces, end="")
        for column in range(width):
            if column == 0 or column == width - 1 or row == 0:
                print("*", end="")
            else:
                print(" ", end="")
        print()

    for row in range(middle - 1, -1, -1):
        spaces = middle - row
        width = 2 * row + 1
        print(" " * spaces, end="")
        for column in range(width):
            if column == 0 or column == width - 1 or row == 0:
                print("*", end="")
            else:
                print(" ", end="")
        print()
```

7. Test Cases

S.No.	Test condition	Expected result
1	Dimension 9	Hollow diamond generated
2	Dimension 8	Invalid dimension displayed
3	Dimension 0	Invalid dimension displayed

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
*
   * *
  *   *
 *     *
*********
 *     *
  *   *
   * *
    *
```

9. Conclusion

The Advanced Numeric Pattern Generator was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 7: PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Validate passwords using minimum length, uppercase, lowercase, digit, special character, repeated-character and prohibited-substring rules. Report every failed rule.

2. Objectives

The main objectives of this case study are:

1. To validate password length and character categories.
2. To detect repetition and prohibited substrings.
3. To explain every failed password condition.

3. Assumptions

The following assumptions are considered:

1. A valid password must contain at least 8 characters.
2. A valid password requires uppercase, lowercase, digit and special-character categories.
3. The words password and admin are prohibited.

4. Algorithm

1. Read a password.
2. Check its length.
3. Traverse its characters and identify uppercase letters, lowercase letters, digits and special characters.
4. Check for three identical consecutive characters.
5. Check for prohibited substrings such as `password` and `admin`.

6. Display every failure or report that the password is valid.

5. Pseudocode

```
START
READ password
failures = empty list
IF length < 8, add length failure
SET uppercase, lowercase, digit and special flags to FALSE
FOR each character
    update the suitable flag
IF any required flag is FALSE, add corresponding failure
FOR each position before the last two
    IF three consecutive characters are equal, add repetition failure
FOR every prohibited word
    IF prohibited word is present, add substring failure
IF failures is empty, display valid
ELSE display all failures
STOP
```

6. Python Source Code

```
password = input("Enter password: ")
failures = []
uppercase = False
lowercase = False
digit = False
special = False

if len(password) < 8:
    failures.append("minimum length is 8")

for character in password:
    if character ⩾ "A" and character ⩽ "Z":
        uppercase = True
    elif character ⩾ "a" and character ⩽ "z":
        lowercase = True
    elif character ⩾ "0" and character ⩽ "9":
        digit = True
    else:
        special = True

if uppercase == False:
    failures.append("one uppercase letter is required")
if lowercase == False:
    failures.append("one lowercase letter is required")
if digit == False:
    failures.append("one digit is required")
if special == False:
    failures.append("one special character is required")

for i in range(len(password) - 2):
    if password[i] == password[i + 1] and password[i] == password[i + 2]:
        failures.append("three consecutive repeated characters are not al
        break

prohibited = ("password", "admin")
for word in prohibited:
    if word in password.lower():
        failures.append("prohibited word found: " + word)

if len(failures) == 0:
    print("Password is valid")
```

```
else:
    print("Password is invalid because:")
    for failure in failures:
        print("-", failure)
```

7. Test Cases

S.No.	Test condition	Expected result
1	Valid complex password	Password accepted
2	Password shorter than 8 characters	Length failure reported
3	Password containing admin	Prohibited substring reported

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Password is invalid because:
- one uppercase letter is required
- one special character is required
- prohibited word found: admin
```

9. Conclusion

The Password Policy Validation Engine was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 8: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Store student marks in a nested list. Calculate each student's total and average, each subject's average, identify the top student and identify subjects whose average is below the required level.

2. Objectives

The main objectives of this case study are:

1. To store student marks in a nested list.
2. To calculate row-wise and column-wise statistics.
3. To identify the top performer and weak subjects.

3. Assumptions

The following assumptions are considered:

1. Every mark is entered as a whole number.
2. Each student has the same number of subjects.
3. A subject average below 50 is considered low.

4. Algorithm

1. Read the number of students and subjects.
2. Store marks in a nested list.
3. Calculate row totals and averages.
4. Calculate column totals and averages.
5. Find the student with the largest total.

6. Report subjects with an average below 50.

5. Pseudocode

```
START
READ students and subjects
CREATE marks matrix
FOR every student
    READ marks for every subject
    STORE row in matrix
FOR every row
    calculate total and average
    update top student if required
FOR every subject column
    calculate column total and average
    IF average < 50, report weak subject
DISPLAY all results
STOP
```

6. Python Source Code

```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))
marks = []
names = []

for i in range(students):
    name = input("Enter student name: ")
    names.append(name)
    row = []
    for j in range(subjects):
        mark = int(input("Enter mark: "))
        row.append(mark)
    marks.append(row)

student_totals = []
top_index = 0

for i in range(students):
    total = 0
    for j in range(subjects):
        total = total + marks[i][j]
    student_totals.append(total)
    average = total / subjects
    print(names[i], "Total:", total, "Average:", round(average, 2))

    if student_totals[i] > student_totals[top_index]:
        top_index = i

print("Top student:", names[top_index])

for j in range(subjects):
    subject_total = 0
    for i in range(students):
        subject_total = subject_total + marks[i][j]
    subject_average = subject_total / students
    print("Subject", j + 1, "average:", round(subject_average, 2))
    if subject_average < 50:
        print("Subject", j + 1, "has low average performance")
```

7. Test Cases

S.No.	Test condition	Expected result
1	Marks for all students entered	Totals and averages calculated
2	Student with highest total	Top student reported
3	Subject average below 50	Weak subject reported

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Ravi Total: 245 Average: 81.67
Meena Total: 190 Average: 63.33
Top student: Ravi
Subject 1 average: 75.0
Subject 2 average: 70.0
Subject 3 average: 72.5
```

9. Conclusion

The Student Marks Matrix Analysis was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 9: TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

1. Problem Statement

Represent employee shifts as tuples containing employee name, day, start time and end time. Detect overlapping shifts for the same employee on the same day.

Times are entered as whole hours, for example 9 for 9:00 a.m. and 17 for 5:00 p.m.

2. Objectives

The main objectives of this case study are:

1. To represent employee shifts using tuples.
2. To compare time intervals for the same employee.
3. To detect overlapping shift assignments.

3. Assumptions

The following assumptions are considered:

1. Time is entered as a whole hour.
2. A conflict exists only for the same employee on the same day.
3. Two intervals conflict when they overlap.

4. Algorithm

1. Read the number of shifts.
2. Store every shift as a tuple `(name, day, start, end)`.
3. Compare every pair of shifts.
4. If names and days match, test whether the intervals overlap.

5. Report conflicting pairs.
6. Use tuple unpacking while comparing records.

5. Pseudocode

```
START
READ number_of_shifts
FOR each shift
    READ name, day, start and end
    STORE (name, day, start, end) in a list
FOR i from 0 to number-1
    UNPACK first tuple
    FOR j from i+1 to number-1
        UNPACK second tuple
        IF names and days are equal AND first_start < second_end AND secon
            report conflict
STOP
```

6. Python Source Code

```
number = int(input("Enter number of shifts: "))
shifts = []

for i in range(number):
    name = input("Enter employee name: ")
    day = input("Enter day: ")
    start = int(input("Enter starting hour: "))
    end = int(input("Enter ending hour: "))
    shift = (name, day, start, end)
    shifts.append(shift)

found_conflict = False

for i in range(number):
    name1, day1, start1, end1 = shifts[i]
    for j in range(i + 1, number):
        name2, day2, start2, end2 = shifts[j]

        if name1 == name2 and day1 == day2:
            if start1 < end2 and start2 < end1:
                print("Conflict:", name1, "on", day1)
                print("Shift 1:", start1, "to", end1)
                print("Shift 2:", start2, "to", end2)
                found_conflict = True

if found_conflict == False:
    print("No shift conflicts found")
```

7. Test Cases

S.No.	Test condition	Expected result
1	Same employee, overlapping times	Conflict reported
2	Same employee, separate times	No conflict for the pair
3	Different employees, overlapping times	No conflict for the pair

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
Conflict: Anu on Monday
Shift 1: 9 to 13
Shift 2: 12 to 16
```

9. Conclusion

The Tuple-Based Employee Shift Conflict Detection was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.

CASE STUDY 10: NESTED LOOP EXAMINATION SEATING GENERATOR

1. Problem Statement

Generate a classroom seating arrangement using row and column identifiers. Some positions are reserved, and alternate seating is required so that adjacent seats in the same row are not allocated together.

2. Objectives

The main objectives of this case study are:

1. To generate a classroom seating arrangement.
2. To preserve reserved positions.
3. To apply alternate seating restrictions using nested loops.

3. Assumptions

The following assumptions are considered:

1. Rows and columns are positive whole numbers.
2. Reserved positions are represented by R.
3. Alternate positions are represented by A and other available positions by -.

4. Algorithm

1. Read rows and columns.
2. Read reserved seat positions.
3. Traverse every position using nested loops.
4. Mark reserved positions as `R`.
5. Mark alternate positions as `A` when `(row + column) % 2 == 0`.

6. Mark the remaining positions as `-`.
7. Display row and column identifiers.

5. Pseudocode

```
START
READ rows and columns
READ number of reserved positions
STORE reserved positions in a list of tuples
FOR each row
    DISPLAY row identifier
    FOR each column
        IF (row, column) is in reserved list
            DISPLAY R
        ELSE IF (row + column) is even
            DISPLAY A
        ELSE
            DISPLAY -
DISPLAY column identifiers
STOP
```

6. Python Source Code

```
rows = int(input("Enter number of rows: "))
columns = int(input("Enter number of columns: "))
reserved_count = int(input("Enter number of reserved seats: "))
reserved = []

for i in range(reserved_count):
    row_number = int(input("Enter reserved row number: "))
    column_number = int(input("Enter reserved column number: "))
    position = (row_number, column_number)
    reserved.append(position)

print("\nSEATING PLAN")
print("    ", end="")
for column in range(1, columns + 1):
    print("C" + str(column), end=" ")
print()

for row in range(1, rows + 1):
    print("R" + str(row), end="  ")
    for column in range(1, columns + 1):
        position = (row, column)
        if position in reserved:
            symbol = "R"
        elif (row + column) % 2 == 0:
            symbol = "A"
        else:
            symbol = "-"
        print(symbol, end="  ")
    print()
```

7. Test Cases

S.No.	Test condition	Expected result
1	Reserved position	R displayed
2	Even row-column position	A displayed
3	Other available position	- displayed

8. Output Screenshots

The following output transcript represents the program output for a sample execution. It can be captured as a screenshot when submitting the printed report.

```
SEATING PLAN
    C1 C2 C3 C4
R1  A  -  A  -
R2  -  R  -  A
R3  A  -  A  -
```

9. Conclusion

The Nested Loop Examination Seating Generator was successfully developed using Python programming concepts. The program applies the required conditions and calculations, processes the given data using suitable lists or tuples where needed, and displays the result in a clear format. This case study demonstrates the use of variables, operators, conditional statements, loops, strings, functions, lists, nested lists, tuples and tuple operations as applicable.